Flight Delay Prediction

Full Pipeline Explanation

Logistic Regression · KNN Imputer · Binary & One-Hot Encoding · Streamlit

EPITA MSc Data Science & Analytics | 2025–2026

Table of Contents

#	Section	Topic
1	Imports	Every library explained with purpose
2	Load Data	Reading CSV, column selection, memory management
3	Clean Data	Filtering cancelled, dropping NaNs, sampling
4	Feature Engineering	DEP_HOUR extraction, binary target creation
5	Train / Test Split	Stratified split, data leakage prevention
6	Binary Encoding	How ORIGIN/DEST are encoded into bits
7	One-Hot Encoding	CARRIER, MONTH, DAY_OF_WEEK encoding
8	KNN Imputer	How KNN fills missing values — step by step
9	Standard Scaler	Why scaling matters for Logistic Regression
10	Logistic Regression	Math, parameters, and training process
11	Evaluation Metrics	AUC-ROC, F1, confusion matrix explained
12	Saving the Model	joblib serialisation for deployment
13	Streamlit App	Every UI element and prediction call explained

SECTION 1

Library Imports

The import section loads every tool we need before any data is touched. Each library has a specific, non-overlapping job in the pipeline. Understanding what each library does — and why it was chosen — is the foundation for understanding the entire project.

Library	What it does in this project
`pandas`	Loads the CSV, filters rows, creates new columns, reindexes column sets. The core data structure is a Data
`numpy`	Used for np.where() to vectorise the target variable creation. Also underpins sklearn internally.
`sklearn.impute.KNNImputer`	Fills missing values (NaN) by averaging the K most similar rows. Placed inside the Pipeline so it only learns
`sklearn.preprocessing.StandardScaler`	Rescales features to mean=0, std=1. Logistic Regression is sensitive to scale differences between features
`sklearn.linear_model.LogisticRegression`	The classification model. Learns a weighted combination of features to predict P(delayed).
`sklearn.model_selection.train_test_split`	Splits data into train and test sets. stratify=y ensures the same delay ratio in both splits.
`sklearn.metrics.*`	Evaluation tools: classification_report (precision/recall/F1), roc_auc_score (ranking quality), confusion_matr
`sklearn.pipeline.Pipeline`	Chains imputer + scaler + model into one object. Prevents data leakage by ensuring scaler is fit only on trai
`category_encoders.BinaryEncoder`	Converts high-cardinality categoricals (ORIGIN, DEST with 300+ values) to compact binary columns (only ~
`joblib`	Serialises the trained pipeline to a .pkl file so the Streamlit app can load it without retraining.
`matplotlib`	Plots and saves the confusion matrix as a PNG image.

SECTION 2 Load Data

The BTS 2015 Airline On-Time Performance dataset contains 5,819,811 rows and over 30 columns. We use **usecols** to read only the 12 columns we need, which reduces memory usage by approximately 70%.

```
cols = ['MONTH', 'DAY_OF_WEEK', 'OP_UNIQUE_CARRIER', 'ORIGIN', 'DEST',
        'CRS_DEP_TIME', 'DEP_DELAY', 'TAXI_OUT', 'AIR_TIME',
        'DISTANCE', 'ARR_DELAY', 'CANCELLED']

df = pd.read_csv(filepath, usecols=cols, low_memory=False)
```

Column	Type	Description	Role
MONTH	int	Month number 1–12	Feature
DAY_OF_WEEK	int	1=Monday, 7=Sunday	Feature
OP_UNIQUE_CARRIER	str	Airline code (AA, DL...)	Feature
ORIGIN	str	Origin airport IATA	Feature
DEST	str	Destination IATA	Feature
CRS_DEP_TIME	int	Scheduled dep. time (HHMM)	Transformed → DEP_HOUR
DEP_DELAY	float	Departure delay (minutes)	Feature (has NaNs)
TAXI_OUT	float	Taxi-out time (minutes)	Feature (has NaNs)
AIR_TIME	float	Actual flight time (min)	Feature (has NaNs)
DISTANCE	float	Distance between airports	Feature
ARR_DELAY	float	Arrival delay (minutes)	Target source → IS_DELAYED
CANCELLED	int	1=cancelled, 0=operated	Filter only

Why low_memory=False? By default, pandas reads CSV files in chunks and guesses each column's data type from the first chunk. On large files, later chunks may contain values that conflict with the guessed type, causing a DtypeWarning and mixed-type columns. low_memory=False reads the entire file before inferring types, producing clean, consistent columns.

SECTION 3

Clean Data

Raw data always contains rows we cannot use. Three cleaning steps are applied before any modelling begins.

```
# Step 1: Remove cancelled flights
df = df[df['CANCELLED'] == 0].copy()

# Step 2: Drop rows with no arrival delay (no label)
df = df.dropna(subset=['ARR_DELAY'])

# Step 3: Sample 200,000 rows for tractable training
df = df.sample(n=200_000, random_state=42).reset_index(drop=True)
```

Step	Why it is necessary
Remove CANCELLED == 1	Cancelled flights never departed. ARR_DELAY is NaN by definition. Keeping them would force us to impute a ta
Drop NaN in ARR_DELAY	ARR_DELAY is the source of our target variable IS_DELAYED. Without a known arrival delay, we cannot creat
Sample 200,000 rows	KNN Imputer has O(n*k) complexity. On 5.8M rows it would take hours. 200k rows trains in minutes while prese
.copy() after filter	pandas returns a "view" of the original DataFrame after boolean indexing. Modifying a view may raise a SettingW

SECTION 4

Feature Engineering

Feature engineering creates new, more informative columns from raw data. We perform two transformations: extracting the departure hour and creating the binary target variable.

4A — Extracting DEP_HOUR

CRS_DEP_TIME stores the scheduled departure time as an integer in HHMM format. For example, 1430 means 14:30 (2:30 PM). The raw integer is not useful as a feature because the model would treat 1430 as numerically close to 1429, but the meaningful signal is the hour of day, not the minute.

```
df['DEP_HOUR'] = (df['CRS_DEP_TIME'] // 100).astype(int)
# 1430 // 100 = 14   (integer division drops the remainder)
# 0845 // 100 = 8
# 2355 // 100 = 23
df = df.drop(columns=['CRS_DEP_TIME'])   # original column no longer needed
```

Why hour matters: Morning flights (6–9 AM) depart on fresh aircraft and have low propagation delay. Evening flights (5–8 PM) accumulate cascading delays from earlier legs. The hour of day is one of the strongest predictors of departure delay in aviation research.

4B — Creating IS_DELAYED (Binary Target)

The FAA and BTS define a flight as 'delayed' if it arrives more than 15 minutes late. This is the official industry threshold. We convert the continuous ARR_DELAY column into a binary label.

```
df['IS_DELAYED'] = np.where(df['ARR_DELAY'] > 15, 1, 0)
# np.where(condition, value_if_true, value_if_false)
# ARR_DELAY = 30  →  30 > 15 is True  →  IS_DELAYED = 1  (delayed)
# ARR_DELAY =  5  →   5 > 15 is False →  IS_DELAYED = 0  (on time)
# ARR_DELAY = -3  →  -3 > 15 is False →  IS_DELAYED = 0  (early = on time)

df = df.drop(columns=['ARR_DELAY'])   # CRITICAL: remove to prevent leakage
```

Target leakage warning: If we kept ARR_DELAY in the feature set, the model would learn the trivial rule 'if ARR_DELAY > 15 then delayed'. This would give 100% accuracy on training data but 0% usefulness in production — you cannot know ARR_DELAY before the flight lands. Always remove the column that directly encodes the target.

SECTION 5 Train / Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,      # 20% of data goes to test set
    random_state=42,    # reproducible split
    stratify=y          # preserve class ratio in both splits
)
```

Parameter	Value	Explanation
`test_size`	`0.2`	20% of rows go to the test set (40,000 rows). 80% (160,000 rows) are used for training. This ratio is stand
`random_state`	`42`	Seeds the random number generator. With the same seed, the exact same rows always go to train/test. T
`stratify`	`y`	Without stratify, random sampling might put 25% delayed rows in train but only 15% in test by chance. str

SECTION 6

Binary Encoding (ORIGIN & DEST)

ORIGIN and DEST each contain 300+ unique airport codes. One-hot encoding would create over 600 new columns — making the model large, slow, and prone to overfitting. Binary encoding solves this.

How Binary Encoding Works

Each unique category is first assigned an integer ordinal. That integer is then expressed in base-2 (binary). The individual bits become separate columns. With N unique values, we need only ceil(log2(N)) columns.

Airport	Ordinal	Binary	Col_0	Col_1	Col_2	Col_3
JFK	0	0000	0	0	0	0
LAX	1	0001	0	0	0	1
ORD	2	0010	0	0	1	0
ATL	3	0011	0	0	1	1
DFW	4	0100	0	1	0	0
...	...	...	...	...	...	...
300 airports	—	—	9 columns total			

```
binary_encoder = ce.BinaryEncoder(cols=['ORIGIN', 'DEST'], drop_invariant=True)

# fit_transform on train: learns the ordinal mapping, then applies it
X_train = binary_encoder.fit_transform(X_train)

# transform only on test: applies the SAME mapping learned from train
# Never fit on test data — that would be data leakage
X_test  = binary_encoder.transform(X_test)
```

fit vs transform: fit() learns parameters (like which airport maps to which ordinal). transform() applies those parameters. We ALWAYS fit on training data only, then apply the same learned mapping to test data. If we fit on test data, the test set 'helps' determine the encoding — this is data leakage and causes overly optimistic evaluation results.

SECTION 7 One-Hot Encoding (CARRIER, MONTH, DAY_OF_WEEK)

One-hot encoding creates one binary column per unique category value. It is appropriate here because CARRIER (~14 values), MONTH (12 values), and DAY_OF_WEEK (7 values) have low cardinality. The total columns added is manageable: 14 + 12 + 7 = 33 columns (minus 3 for drop_first).

```
# Convert to string so get_dummies treats them as categories
for col in ['MONTH', 'DAY_OF_WEEK']:
    X_train[col] = X_train[col].astype(str)
    X_test[col]  = X_test[col].astype(str)

X_train = pd.get_dummies(X_train, columns=ohe_cols, drop_first=True)
X_test  = pd.get_dummies(X_test,  columns=ohe_cols, drop_first=True)

# Align: test might be missing columns that train has
X_test = X_test.reindex(columns=X_train.columns, fill_value=0)
```

Concept	Explanation
Why convert MONTH to string?	If MONTH stays as integer, get_dummies treats it as a numeric column and does nothing. As a string, it becom
drop_first=True	With 7 day-of-week columns, if we know it is not Mon/Tue/Wed/Thu/Fri/Sat, it must be Sun. The 7th column is p
reindex(fill_value=0)	If the test set has a carrier that only appeared in training (or vice versa), get_dummies creates different column

SECTION 8

KNN Imputer — Filling Missing Values

Three columns have missing values: DEP_DELAY, TAXI_OUT, and AIR_TIME. These are NaN when a flight's data was not recorded. We use KNN Imputation — a strategy that fills each missing value using information from the most similar rows in the dataset.

How KNN Imputation Works — Step by Step

Step	Description
1. Identify missing cell	For each row with a NaN in (e.g.) TAXI_OUT, we need to estimate its value.
2. Find K nearest neighbors	Using all other (non-missing) features of that row, compute the Euclidean distance to every other row in the data
3. Aggregate neighbor values	Take the average TAXI_OUT value from those 5 neighbors.
4. Fill the missing value	Replace the NaN with that average. The result reflects local patterns rather than the global mean.

```
# KNNImputer is placed INSIDE the Pipeline as step 1
# This ensures it is only fit() on training data
('imputer', KNNImputer(n_neighbors=5))

# During pipeline.fit(X_train, y_train):
#   KNNImputer.fit(X_train)           — learns the neighbor structure
#   KNNImputer.transform(X_train)     — fills NaNs in training set

# During pipeline.predict(X_test):
#   KNNImputer.transform(X_test)      — fills NaNs using training neighbors
```

KNN vs Mean Imputation: Mean imputation replaces every NaN with the global column average — a short 50-mile flight and a 3,000-mile flight get the same imputed AIR_TIME. KNN finds similar flights (short routes, same carrier, same distance range) and uses their AIR_TIME instead. This preserves local structure and improves model quality.

SECTION 9

Standard Scaler — Feature Scaling

Logistic Regression optimises its weights using gradient descent. If features have very different scales, the gradient updates are disproportionate — the optimiser converges slowly or not at all. StandardScaler transforms each feature to mean=0 and std=1.

The Formula

```
x_scaled = (x - mean(x)) / std(x)

# Example: DISTANCE column
# Original range: 50 to 5,000 miles  (mean ≈ 850, std ≈ 600)
# After scaling:  -1.33 to +6.9      (mean = 0, std = 1)

# Example: MONTH column
# Original range: 1 to 12            (mean ≈ 6.5, std ≈ 3.5)
# After scaling:  -1.57 to +1.57     (mean = 0, std = 1)
```

Why this matters for Logistic Regression: The model learns one weight per feature. Without scaling, the gradient for DISTANCE (large values) will be tiny (to keep predictions bounded), while the gradient for MONTH (small values) will be huge. The optimiser treats them inconsistently. After scaling, all features contribute on equal footing.

SECTION 10

Logistic Regression — The Model

Logistic Regression is a linear classifier that estimates the probability of a binary outcome. Despite its name, it is a classification algorithm, not a regression algorithm.

The Math

```
# Step 1: Linear combination of features
z = w0 + w1*DEP_DELAY + w2*DISTANCE + w3*DEP_HOUR + ... + wn*feature_n

# Step 2: Sigmoid function — squashes z to probability between 0 and 1
P(delayed) = 1 / (1 + e^(-z))

# Decision rule:
# If P(delayed) > 0.5  →  predict 1 (delayed)
# If P(delayed) <= 0.5 →  predict 0 (on time)

# The model learns weights w0...wn by minimising log loss:
# Loss = -[y*log(p) + (1-y)*log(1-p)]
```

Parameter	Value	Why this value
`max_iter`	`1000`	Maximum gradient descent steps. Default is 100, which often fails to converge on large, high-dimensional
`C`	`1.0`	Inverse of regularisation strength. Low C = strong regularisation (simpler model, may underfit). High C = w
`solver`	`lbfgs`	The optimisation algorithm. L-BFGS is a quasi-Newton method that approximates the Hessian (second de
`class_weight`	`balanced`	The dataset has ~80% on-time and ~20% delayed. Without this, the model ignores the minority class. bal
`random_state`	`42`	Seeds the internal random number generator for reproducibility. With the same seed, training always proc

SECTION 11

Evaluation Metrics

Metric	Formula	What it means for this project
Precision	**TP / (TP + FP)**	Of all flights predicted as delayed, what fraction were actually delayed? Low precision = many fa
Recall	**TP / (TP + FN)**	Of all actually delayed flights, what fraction did we catch? Low recall = many missed delays (pas
F1 Score	**2 * (P*R) / (P+R)**	Harmonic mean of precision and recall. Balanced metric when classes are imbalanced. Use this
AUC-ROC	**Area under ROC curve**	Probability that the model ranks a delayed flight higher than an on-time flight. 0.5 = random, 1.0
Confusion Matrix	**[[TN, FP], [FN, TP]]**	Full breakdown of correct and incorrect predictions by class. Shows exactly where the model ma

SECTION 12

Saving the Model with joblib

After training, we save the entire pipeline and all encoding artifacts to a single .pkl file. This allows the Streamlit app to load a ready-to-use model without retraining from scratch on every launch.

```
joblib.dump({
    'pipeline':       pipeline,        # KNNImputer + Scaler + LogReg
    'binary_encoder': binary_encoder,  # fitted ORIGIN/DEST encoder
    'feature_names':  feature_names,   # exact column order model expects
    'ohe_cols':       ohe_cols,        # which columns were one-hot encoded
    'binary_cols':    binary_cols,     # which columns were binary encoded
}, 'flight_delay_model.pkl')

# Loading in the Streamlit app:
artifacts = joblib.load('flight_delay_model.pkl')
pipeline = artifacts['pipeline']
```

Why save all artifacts together? At prediction time, the Streamlit app receives raw user input (airport codes, carrier, etc.). It must apply the exact same encoding steps as training. If we only saved the pipeline (which handles imputation and scaling), but forgot the binary_encoder, ORIGIN and DEST would not be encoded and the prediction would fail.

SECTION 13

Streamlit App — User Interface

The Streamlit app is the user-facing interface. It loads the saved model, accepts flight details through interactive widgets, applies the same preprocessing as training, and displays the delay prediction with probability.

How to Run the App

```
# In your terminal, from the project folder:
streamlit run app.py

# Streamlit will open a browser tab at:
# http://localhost:8501
```

Component	Streamlit Function	Purpose
Page config	`st.set_page_config()`	Sets browser tab title, icon, and layout width
Model cache	`@st.cache_resource`	Loads the model once; reuses it on every interaction
Title	`st.title()`	Large page heading at the top
Two-column layout	`st.columns(2)`	Places input widgets side by side
Dropdown	`st.selectbox()`	Month and day-of-week selection
Text input	`st.text_input()`	ORIGIN and DEST airport code entry
Slider	`st.slider()`	Departure hour (0–23)
Number input	`st.number_input()`	Departure delay, distance, taxi-out, air time
Predict button	`st.button()`	Triggers the prediction when clicked
Metric display	`st.metric()`	Shows delay probability as a large number
Progress bar	`st.progress()`	Visual bar showing probability 0–100%
Result box	`st.error() / st.success()`	Red for delayed prediction, green for on time
Expander	`st.expander()`	Collapsible section showing the raw input data
Sidebar	`st.sidebar`	Panel showing model information and project details

Prediction Flow Inside the App

```
# 1. Collect user inputs into a 1-row DataFrame
input_df = pd.DataFrame({'ORIGIN': ['JFK'], 'DEST': ['LAX'], ...})

# 2. Apply binary encoding (same encoder fit during training)
input_enc = binary_encoder.transform(input_df)

# 3. Apply one-hot encoding
input_enc = pd.get_dummies(input_enc, columns=ohe_cols, drop_first=True)

# 4. Align columns to match training feature set exactly
input_enc = input_enc.reindex(columns=feature_names, fill_value=0)

# 5. Run pipeline (impute → scale → predict)
delay_prob  = pipeline.predict_proba(input_enc)[0][1]  # P(delayed)
prediction  = pipeline.predict(input_enc)[0]            # 0 or 1

# 6. Display result
st.metric('Delay Probability', f'{delay_prob:.1%}')
st.error('Flight likely DELAYED') if prediction == 1 else st.success('ON TIME')
```

SECTION 14 Project File Structure & Setup

```
flight_delay_project/
■■■ pipeline.py                  # Training pipeline (run this first)
■■■ app.py                       # Streamlit app (run after training)
■■■ flights.csv                  # BTS dataset (download from Kaggle)
■■■ flight_delay_model.pkl       # Saved model (created by pipeline.py)
■■■ requirements.txt             # Python dependencies
```

```
# requirements.txt
pandas>=2.0
numpy>=1.24
scikit-learn>=1.3
category_encoders>=2.6
joblib>=1.3
streamlit>=1.28
matplotlib>=3.7

# Install all dependencies:
pip install -r requirements.txt

# Run training pipeline:
python pipeline.py

# Launch Streamlit app:
streamlit run app.py
```

Dataset download: Go to https://www.kaggle.com/datasets/usdot/flight-delays and download the zip. Extract flights.csv into your project folder. The file is ~571 MB. Alternatively, use the BTS direct download at https://www.transtats.bts.gov/DL_SelectFields.aspx?gnoyr_VQ=FGJ